HCI Project (Vietnam)

Emanuele Zini, Fengyi Zhu, Iiris Atti

July 30, 2023

1 Executive Summary

During the development of the project we followed the Gantt diagram provided in the lectures. We split the workload based on each student's skill set. We have a multidisciplinary team so the split was natural. Instead of quantity, we opted for quality and achieved a nice final product that is in line with the users' needs. We came up with this product due to our group member's personal experiences with the parking lot and its issues. Emanuele Zini described the situation in the following way: "*I'm myself a user that park in the university parking and I recognize by my own that the parking has a lot of problems that should be resolved and the park shouldn't be left to itself and like a "farwest", so I felt the need of doing something to change this situation.*"

We started with the benchmarking phase and browsed through already existing parking hardware and applications. Based on the results, we created the design library. Then, we moved to the data gathering and the PACT analysis. We created a questionnaire. To deepen our understanding on the issues, we interviewed people regarding the parking lot. From this data we derived the requirements. Also, the interviewees seemed to be very optimistic and open to our description of the future product.

After we confirmed the user's needs we created a sketch of the application. Our goal was to create an application that is simple and fast to use. It should have the most important features. The users want to be able to make sure that there is space for their car before leaving. They want to be able communicate issues regarding the parking area. We also differ from the other parking applications by offering a custom solution according to the University parking lot's needs. We don't use sensors, which makes the solution cost effective. We assign parking spots to each user with the help of AI, since the parking lot lacks lined spots.

After designing the prototype, we started the evaluation phase. We used Heuristic Evaluation. We designed the final medium-fidelity prototype based on the evaluation results. With even more iterations the outcome could have been better. However based on the user testing, users seemed to be satisfied with the medium-fidelity prototype.

2 Analysis of existing products and ideas generation

2.1 Existing products

Parking is becoming more and more digital. Smart parking systems offer seamless digital payment, together with maximal usage of the available parking space. An effective parking

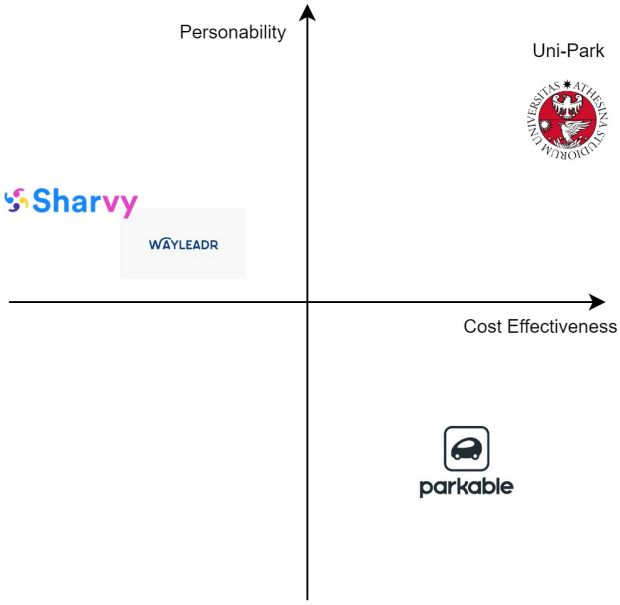
system not only saves user's time, but also improves the parking experience by making parking easy and comfortable.

The goal of this benchmarking is to study the most used parking hardware and applications. The studied parking applications are limited to those offered to certain parking facilities, since our solution will focus on a single parking lot. Next we will discuss the most used sensors in parking solutions.

Different parking sensors for vehicle detection are used in parking spaces. The most common ones are the overhead indicator sensors, in-ground sensors and surface-mount sensors. At best, sensors coupled with digital signage system (in this case LED guidance sensors) help visitors to find available parking 37% faster than without the sensors.

Overhead indicator sensors detect the vehicle with cameras that are installed above each parking stall and indicate the availability of the stalls to the drivers. This type of sensor is typically paired with a digital signage and license plate recognition system. The data can be further sent to a central management system for, *e.g.*, business analysis. The in-ground sensor is installed below the ground, detects the vehicle, and communicates it to the central management system. The surface-mount sensor works similarly but is fixed to the road surface.

There are multiple applications that aim to simplify the user's parking experience. Since we are focusing on making a parking solution for the University's parking lot, the most important applications in our case are such that offer an application for a certain facility. These kinds of applications are, *e.g.*, **Sharvy**, **Wayleadr** and **Parkable**.



Sharvy is an online platform that consists of an application and a web page. It aims to optimize the occupancy of the customer's shared spaces, such as a car park, work stations and/or a cafeteria. It offers a booking system with a real-time view of the availability of space, usage statistics, car park customization and access control.

Wayleadr offers an application with similar features as Sharvy, with the addition of a feature of an EV charging management. The company also markets itself as solely a smart car park system, unlike Sharvy. The UI is intuitive with only one accent color. Furthermore, the usage of color and opaqueness in the design is good and guides the user's attention to the right direction. The parking capacity is easy to read and is expressed in percentages.

Parkable has otherwise similar features as Sharvy and Wayleadr with the exception that

it provides a possibility for partially public parking, *i.e.,* guest parking. In addition, it offers an automated admin with automatic problem resolution, tracking and reports. Users access the space via Bluetooth authentication or ANPR recognition. The payment happens through the application. Parkable has many parking lots that are available for users. It offers a possibility to book a space for a short or a long stay. The UI is otherwise clear, but lacks loading information; the user does not know if there is no parking available or if the application is just loading. It also lacks an explanation of the difference of casual parking and long-term parking, which may be confusing to users.

There are various popular applications with a similar idea, such as EasyPark, ParkMe and SpotHero. Thus, it is difficult to make innovations in the field. However, we cannot use these applications to solve the issue with the parking lot of the University of Trento, since there are no lines that would mark off the spots for parking. Furthermore, we need to apply an access control that only let's in people that are associated with the university. Also, having a self hosted application is cost wise better than having a third party software. In a custom solution we can provide personalized extra features. We could for example enable direct communication with the university staff and the users. The university staff could, for instance, personally contact a user whose car is badly parked. Moreover, for the university a one time payment solution would be cheaper compared to a long term contract.

It was difficult to analyze the applications, since they require you to be a paying user to view the full product. link of the benchmarking presentation.

2.2 Idea generation and design space

Based on the benchmarking, we wanted to design an application that works together with overhead sensors (cameras). The application should be easy to use, have the possibility to book a parking spot and see the real-time occupancy. We will discuss the features more in Section 3, in which we present the data gathered from the participants.

The system should also have access control, for example so that the parking would be possible only to people with a UniTrento-email address. This could be done via a QR-code and a vehicle access gate (a beam) with a QR-code scanner.

Since there are no lines to mark the spots, we need an algorithm that assigns a spot to the user according to the availability of the lot. The application should also alert the user in case they have not parked properly. The users should be able to report any problems within the parking lot to the management. link to the design library presentation.

3 PACT analysis

3.1 Data gathering techniques

We started the data gathering by creating a Google Forms (link) with general questions about the user satisfaction in the parking lot. We included questions about what type of features the users would find useful and need in a parking application. Our goal with the questionnaire was to find out, what the main issues of the university parking lot's users have. The participants are people who use the parking lot, mainly students. Before sharing the questionnaire, we performed a pilot study and modified the questions according to the results. We put out QR-codes of the questionnaire and set them on the windows of cars parked on the parking

lot. In total 21 people answered the Google Forms. The number might seem small, but if we consider the total amount of people who frequently use the parking, 21 is still a decent amount for giving reflections and data of the overall users. We started data gathering with the questionnaires in order to find out all possible issues regarding parking. To further deepen our understanding of the issues, we also conducted interviews (link)

Most of the people who filled in the questionnaire were male university students. Clear majority (90.5%) of the respondents are 18–30 years old. A majority of the respondents (46.2%) use the parking lot 1–3 times a week, 30.8% less than once a month and 15.4% use it 3–7 times a week. The rest (7.7%) use it 1–4 times a month. Most of the respondents are drivers (around 70%), thus not too many people come to the parking lot using the same car. The median of satisfaction in the condition of the parking lot is 2 and the average is 2.46. The median satisfaction in the management of the parking lot is 2 and the mean is 2.38. The answers' scale ranges from 1 to 5, 1 being not satisfied and 5 highly satisfied. Thus, the users are generally unsatisfied with the condition and management of the parking. The most reported problems in the parking are 'potholes', 'ice in the winter', 'badly parked cars and issues with entering or exiting the parking lot' and 'dirt while parking'. In addition, there was one mention of not enough space in the parking lot. An interviewee described their issues with the parking in the following way: "*I think the main problem is the fact that the university cannot intervene to ask to move the cars blocking the passage, as happens in Povo 1. I was still blocked due to people who park very badly and, when I called the reception desk, I was always told that they couldn't do anything about it.*"

It is clear that there is room for improvement within the parking conditions. In order to find out what features the potential users of the application would find useful, we included questions of them. The features that we suggested were: **'information of occupancy'** (how many parking spots are in use), **'assignment of a spot'** (the application gives the user a spot where to park), **'book a spot beforehand'** (a reservation system), **'report a problem'** (report a problem to other users, this could be posted in the application), **'opening hours'**, **'communication of parking'** (communication of sudden or exceptional changes in the parking lot, such as temporary closures), **'heatmap'** (a graph of the occupation of the parking lot over time), **'consideration of vehicle size'** (the application would assign a spot considering the size of the vehicle) and finally the **'location of the parked car'**. Figure 1 includes violin plots of the interviewees' answers. The scale is again from 1 to 5, 1 being not important and 5 highly important.

Based on the violin plots, it is clear to see that the most important application feature according to the interviewees is the **'communication of parking'**. The most important features after that seem to be **'information of occupancy', 'report a problem', 'opening hours', 'assignment of a spot'** and **'book a spot beforehand'**. On the other hand, the users did not find the features **'consideration of vehicle size'** and **'location of the parked car'** important. The feature **'heat map'** falls quite in the middle with a 2.69 average and median 3. The most important thing to the user seems to be that they know that **there is room for their car before arriving to the parking lot** and that they **can communicate issues regarding the parking area**.

We conducted interviews to understand what the user feels during parking. The interviewees were unsatisfied with the condition of the parking. They complained about the University's lack of taking action in the condition. Some of them also described the parking experience as stressful and mentioned, that they had been late to lectures many times because of the poor

organization of the area. In general, the interviewees liked the idea of a booking system in the parking lot. One interviewee had an essential contribution to our final product by expressing their worries of our product's entrance mechanism; we were supposed to only allow entrance to users with our application. However, not all people have time or memory to download it. This is why we also grant access to users with the university service card.

Figure 1: Violin plots of the possible features of the application. The answers range from 1 to 5, 1 being not important and 5 highly important. The data was gathered via a questionnaire. Code for creating the plot.

People	**Primary**: University Students, attendees, employees **Secondary**: passengers of the primary users **Tertiary**: the University rector
Activities	**Pragmatic/hedonic**: activity mainly pragmatic **Regular/infrequent**: depends on how regularly the user parks in the parking lot. Activity can be both **Continuous/interrupted**: interrupted, since users can use the product whenever it is needed **Cooperation**: there is none **Complexity**: the application is designed to be easy to use and the actions that you do with it are not complex (booking a spot, check opening hours etc.)
Context	**Physical**: at home before going to the university and in the car **Social**: the activity is done individually **Psychological**: stressful due to time pressure and schedules
Technologies	**Input**: GPS, cameras **Output**: screen, QR-code, numerical code (used if the QR does not work) **Communication**: communication with the server via Wifi or mobile data, satellite connection (GPS)

3.2 Requirements

The requirements are based on the data that was presented in Section 3. We also created user stories.
The requirements can be divided into functional and non-functional requirements. Link to the list of all the requirements.

Functional requirements:

- "The system should enable clear communication between the users and the personnel responsible for the parking lot maintenance."

- "The system should reserve and assign a parking spot for the user."
- "The system should monitor and communicate the real-time occupancy of the parking lot."
- "The system should provide the user with all available information of the parking lot, e.g., temporary closures or other exceptional occasions."
- "The user should be able to use the parking lot if there are free spots and they don't have the application downloaded"

Non-functional requirements:

- "Clear consistent interface that is easy to navigate."
- "The system should respond quickly and give accurate real-time information to the users.
- "The system should work in an unmaintained parking space and assign spots considering the current situation. (This would be conducted using AI.)"
- "The system should be quick to use, since users should be able to use it even in a hurry."

4 1st Iteration

4.1 Design methodology

4.1.1 Personas

Matteo, 55, University Professor at the University of Trento. Matteo is a busy man who works as a professor and researcher for the University. He likes to plan his daily schedule beforehand to make sure he finishes his duties on time (**attitudes and motivations**). He likes driving and has been described a good driver (**dis/abilities**). He has a smartphone (and a UniTrento-email) and is quite experienced in several applications (**technology proficiency**). He has a car and drives to work every day (**resources**). He has to make it to work on time, but his mornings are usually busy since he also drives his children to school before his work (**personal context**). **KEY NEEDS:** *I would like to reach my workplace on time and waste as little time as possible in finding parking spot for my car.*

Giulia, 20, University Student at the University of Trento. Giulia is a Bachelor's Student who works part-time. She does not like scheduling and planning her days (**attitudes and motivations**). Giulia is spontaneous and her time management skills are not excellent (**dis/abilities**). She has a smartphone (and a UniTrento-email) and is comfortable using her phone and various applications (**technology proficiency**). She has a driving license but does not own a car. However, she sometimes borrows her parents car if she has a lecture right after work. (**Resources**). Her work shifts are irregular and sometimes she has to work before lectures. Thus, her schedule is sometimes hectic (**personal context**). **KEY NEEDS:** *I would like to get to my lectures on time even if I am working before school. I would like to be sure that I have an easily accessible place for my car.*

Alessandra, 60, part of University Maintenance Personnel. Alessandra has worked at the University of Trento for 20 years. She is a conscientious worker who loves interacting with people (**attitudes and motivations**). She does not like driving and is unfamiliar

with parking lots (**dis/abilities**). She has a smartphone, but struggles with using applications (**technology proficiency**). She has a driving license and a car, but she does not like using it (**resources**). She faces a lot of complaints about the condition of the University parking lot. She sometimes comes to work with a car with her son, who also works for the University (**personal context**). **KEY NEEDS:** *I would like to focus on work duties that I can help with. It would be great not to get complaints about things that are out of my control.*

Link to the user personas presentation.

4.1.2 Scenarios

Matteo's child did not hear their alarm in the morning and slept too late. Matteo has to drive them to school and he has to wait for the child to get ready. He is running late for work and is worried about not getting a parking spot for his car. He downloads our parking application and sees the current occupation of the parking lot. He books a spot via the application and gets a QR code for the booking. By showing this QR code at the parking lot to a QR code -scanner, a parking control beam opens and lets Matteo into the parking lot.

Before the parking application: Alessandra is working and a student who has parked their car at the parking lot comes to ask for help. Their car is stuck since other people have parked their cars carelessly, leaving hers stuck between theirs. Alessandra apologises and tells that there is nothing she can do. The student gets angry and Alessandra has to calm them down. This wastes her time since she cannot do her own work tasks. **After the parking application:** The application assigns the parking lot users their own spots. This way cars will not get stuck behind each other and Alessandra can focus on her other tasks and not deal with parking lot complaints.

Massimo is driving to the University. His smartphone's storage is full and he is unable to download the parking application. He sees that there is still room at the parking lot. The spots are unoccupied and not reserved. He uses his university service card to access the parking area. The service card works as an access card. He swipes the card at the entrance. The parking control beam opens and he drives in and parks the car.

Link to all of the scenarios.

4.1.3 Interface

We took inspiration for the prototypes from the benchmarking phase. The most important features in parking application are the booking and occupation features. The application should be quick to learn. In order to support the learnability of the system, we decided to implement familiar system metaphors (such as a car symbol in the menu for parking and speech bubbles for feedback).

Since the application concerns the University parking area, the theme should be similar to other software and applications used by the University (e.g., Moodle and UniTrentoApp). Thus, the main colors that we use are red and white. The booking page is also similar to that of the UniTrento App.

In order to use the parking lot without the application, we have the option to use the university service card as an access token.

4.2 Prototyping (low-fidelity)

In the low-fidelity prototype we aimed to design a fully functional app for ease of use and navigation. With a simple and user-friendly navigation system, the user can reserve a parking space with few button presses even in short time and stressful situations.

Figure 3 is our low-fidelity prototype. The prototype begins with a loading/opening page

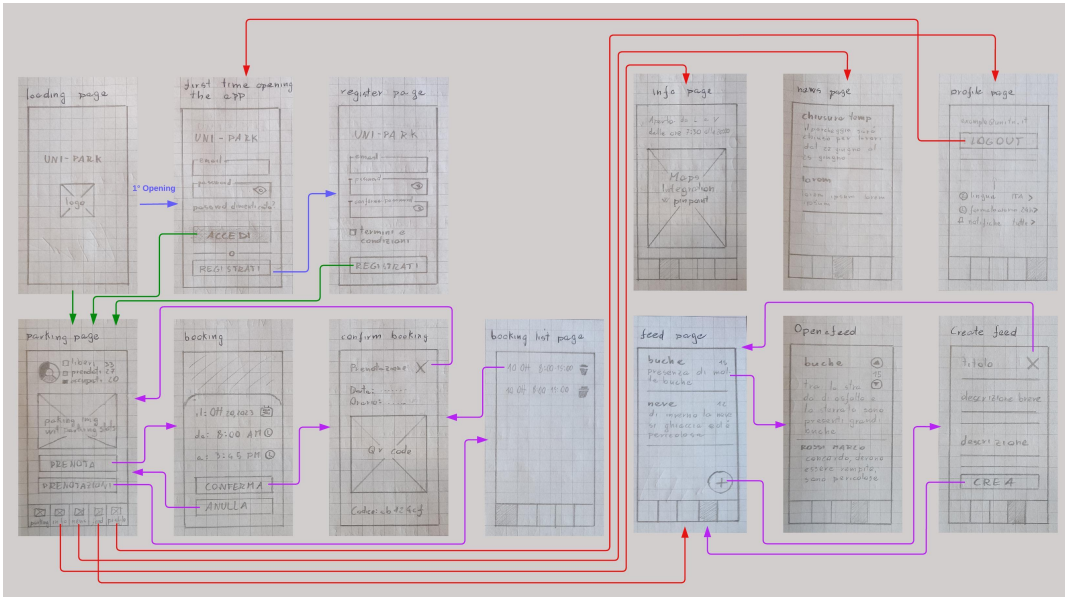


Figure 2: Low-fidelity Prototype. High definition image

with the logo of the application. After loading, the user is directed to the login/registration page. The user registers with their UniTrento-email. The login registration page is showed only the first time, after that it will load to the parking page.

After the user has signed in, they are directed into the main page of the application. The main page includes information of the occupation of the parking lot and a menu bar below the page. The menu bar contains a section for parking, general information, news, feed and profile. In addition, there are buttons for booking a parking spot and a button for booking list.

Using the booking button, the user moves to the booking page. Here the user can book parking for a certain day at a certain hour, according to the parking lot's availability. After choosing a time slot the user may either confirm the booking or cancel their actions. If they press the button "cancel" they are redirected into the main page. If they press "confirm", they get a QR-code and a numerical booking code that are used at the parking lot access. The parking control beam (that we would implement) opens when a correct QR-code or numerical code are either scanned or tapped at the access point.

The user can access a list of bookings from the main page. Here they can view their upcoming bookings and delete them. By pressing a booking they are directed into the booking confirmed-page, where they see the QR-code and the numerical code.

The user can access a general information page via the main page's menu bar. This page contains information about the parking lot, such as the opening hours and a map of the area.

Another page accessed via the menu bar is the news page. It contains news of the parking lot, such as temporary closures.

By pressing the menu bar's feed button, a feed page opens. Here is displayed a list of comments left by other users of the application. By pressing an item in the feed, the user sees the whole text and who posted it. The user can also create their own post on the feed by pressing "+"-button.

The menu bar includes a "profile"-button. Here the user can edit their profile and change their application settings. Language, time format and the amount of notifications can be modified here. The page also contains a log out button. When this button is pressed, the user gets directed into the registration/login page.
Link to the low-fidelity prototype.

4.3 Evaluation

4.3.1 Heuristic Evaluation

Visibility of system status: 1.There could be more "back" buttons and put titles in every pages so that it is clear for the user on where in the system hierarchy they are in and how they can get back to where they once were. 2.There could be notifications with visual or audio feedback of changes in the system, e.g., when someone posts a new post or there's new information of opening hours or when the booked time slot is about to start or a badge notification on news tab if there are new news to keep the user always informed. 3.There could be feedback when the user after each interaction and also feedback and confirmation when the user has booked a parking spot and after the users post something on the "create feedback"page. 4.When deleting a booking or confirming a booking, put an screen notification of the result of the operation with success or error message.

Match between system and the real world: 1.There should add appropriated icons for the bottom navigation making it easier for user the navigation.2.In the medium fidelity prototype, the map of the parking lot should correspond to the parking lot in the real world.

User control and freedom: 1.There should be an option to cancel the booking on the page "booking" and button "annulla". 2.There should be also an option to delete user's own posts somewhere on the feed page.

Consistency and standards: 1.The app should follow established industry conventions.

Error prevention: 1.In the booking page default the values to the current day or the day after if is late in the day, prevent users for selecting dates prior to the current day. 2.In the booking list page make the icon for deleting the booking disabled (with disabled styles) to not lest user delete past booking already used. 3.There should be an undo button to avoid user cancel the booking by mistake.

Recognition rather than recall: 1.The app should show the expired booking information. 2. There should be information on how many free spots there are when selecting a date in the booking page.

Flexibility and efficiency of use: 1.A swipe gesture can be implemented in the book-

ing list page to delete reservation without clicking the icon. 2. When the user enter a certain keyboard in the feedback there should be a function to automatically associates relevant previous question.

Aesthetic and minimalist design: 1.In the parking page the "prenota" button should be designed as primary action button while the "prenotazioni" as secondary one to keep the main focus on create a new reservation.

Help users recognize, diagnose, and recover from errors: 1. there should be notifications describing the message when the user try to do an action and it fails (like: no internet connection or wrong password). 2.There should be clear error messages, for example in case the user inserted the wrong password or email address, if the booking was not successful, if a post could not be uploaded to feed.

Help and documentation: 1.There should be some sort of simple help documentation available. The user should have the opportunity to also contact people in charge of the system, and the parking maintenance personnel etc.

According to the low-fidelity prototype evaluation, we think it gets a clear and user-friendly interface, but we still have to do a lot of changes in medium-fidelity prototype. We proposed a large number of revisions in the section of visibility of system status and error preventionWe need more interaction buttons to let users clearly understand where they are on the interface. We also need to add more prompt boxes to prevent users from deleting certain information and reservations by mistake. Consider the problem from the user's point of view to make the users get a better experience.

Link to the complete Heuristic Evaluations.

4.3.2 User testing

Fort the low-fidelity prototype user testing, we invited three tester and asked them to complete the various tasks, since the low-fidelity prototype was made of paper we just switched the pages when they tapped anywhere on the paper page.At the end of the task the users participated in a short interview and it was discussed how they felt using the prototype, if they felt confused or at ease navigating in it, and what they think should be changed or that can be improved.

With the tasks, we designed it into 10 steps: 1) create an account 2) create a booking for Tomorrow from 8:00 to 15:00 3) show the code of the last booking 4) delete the last booking 5) find at which hour the park is open 6) change the app Language 7) check the last news about the parking 8) create a feedback of a problem that the parking have 9) up-vote the feedback of another user 10) logout from the app

The task is to test the create account system, booking system, information page, feedback system and the profile page. During the testing, every tester completed the task very fast and without any problem, they think the application is intuitive and easy to use. Only thing is that a user a bit of indecision on task 9 to where to tap for the up-vote.

Link to the user testing.

5 2nd Iteration

5.1 Design

Inspired by the low-fidelity prototype, we added more user interaction details to the medium-fidelity prototype, such as the news module in the navigation bar of the application. Prompt and Quantity Information for Messages. In the part of error prevention, we have added more error prompt windows, such as prompts for user input errors, prompts for account non-existence, prompts for users who have scheduled unavailable time, etc. In the navigation bar of the main interface, we have added more user-friendly icons to make the interface more concise and clear. Each interface adds an interface title, so that users know more about the page they are in. A confirmation interface has been added to the deletion reservation interface, which prevents users from deleting reservation information by mistake. In the parking lot information page, we updated the real map location of the parking lot, which can be zoomed in and out, making it easier for users to find the location. In the mid-fidelity prototype, we added more back buttons to make the interface order clearer. Here is a video of the prototype in action, that could be found on drive, and also the Axure prototype

5.2 Prototyping (medium-fidelity)

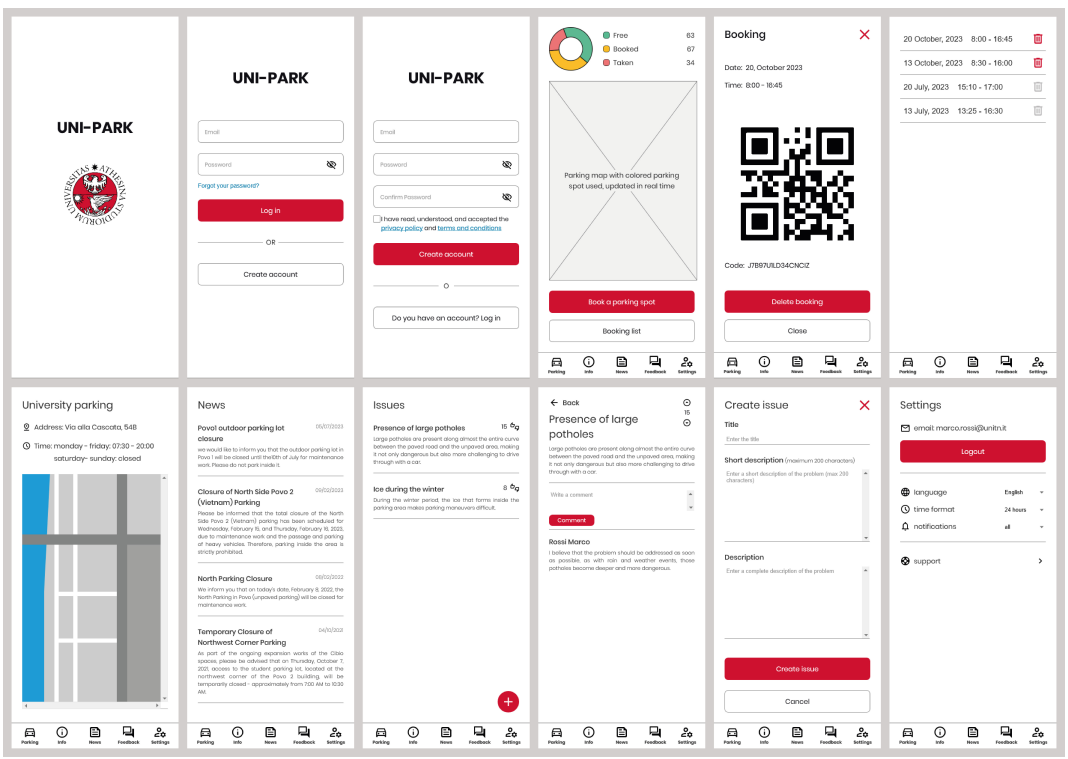


Figure 3: Mid-fidelity Prototype. High definition image

5.3 Evaluation

In the evaluation of mid-fidelity prototypes, We invited 3 testers sending the mid-fidelity prototype link to the them and asked them to perform the operations that we organized in their own mobile phone. When they finished testing, we made a short interview and asked them how the experience was like.

Our testing apart is dividing into fourteen steps: 1) create an new account. 2) create a booking for 1st August from 9:30 am to 11:00 am. 3) show the code of the booking. 4) delete the booking. 5) find the booking list and delete the first booking. 6) find at which hour the park is open. 7) find the parking lot address from the map. 8) check the last news about the parking. 9) check one of the existing feedback, enter it and up-vote it. 10) create a feedback of a problem that the parking have. 11) change the app Language. 12) change the time format. 13) change the notification form. 14) logout from the app. The first step is to let the users test the creating account system,whether the user can create an account successfully. The second to fifth step is to testing the booking system to see whether users can create a booking and get the QR code also can manage deleting it. The sixth and seventh step are testing the parking lot's information page to see whether the users can find the parking lot's address properly. The eighth step is testing the news page. The ninth and tenth steps are testing the feedback system part, and the last four steps are testing the profile page, whether the user can make a change in the app language, time format and notification form and then manage to logout the system.

All of the tester complete the test very fast and successfully without any mistakes. The first tester considers the app is very user-friendly, especially the information page's map which can zoom is a very successful detail because she thinks it can guide new people to find the place very fast. The second tester likes the color design and the navigation bar, he describes it as a very intuitive thing and believes the app will be useful in the parking lot. The last tester likes the simple operation and the fully function, she says this app has all the functions she need. Overall, this user testing we had a very positive feedback.

Link to the user testing.

6 Conclusions

In the four months of learning, from the initial project topic selection to finally achieving tangible results, it has been a challenging journey. Throughout these months of work, we have deeply understood that the essence of human-computer interaction systems lies in prioritizing the human aspect. Also we learned how every step is a key factor in creating a working product, and since every step is based on the step before, just skipping one and doing assumption without checking them with data gathering will lead to a probable product fail.

Starting with the collection of user requirements and integrating them to determine the necessary functionalities of the application, we created the first version of the low-fidelity prototype. We conducted extensive user testing using the low-fidelity prototype, collected valuable feedback from users, and iteratively updated the system functionalities, simplified the interface design, and addressed issues. User feedback has been the guiding principle in our system updates.

Throughout the learning process, we also realized the importance of teamwork. The quality of teamwork heavily relies on effective communication among team members. Conflicts among team members are inevitable, and learning how to properly resolve these conflicts is a crucial aspect we have learned.

Ultimately, we successfully completed the creation of the high-fidelity prototype. We firmly believe that this application will bring convenience and joy to users. We look forward to continuing to optimize and expand it in the future, creating even better user experiences.